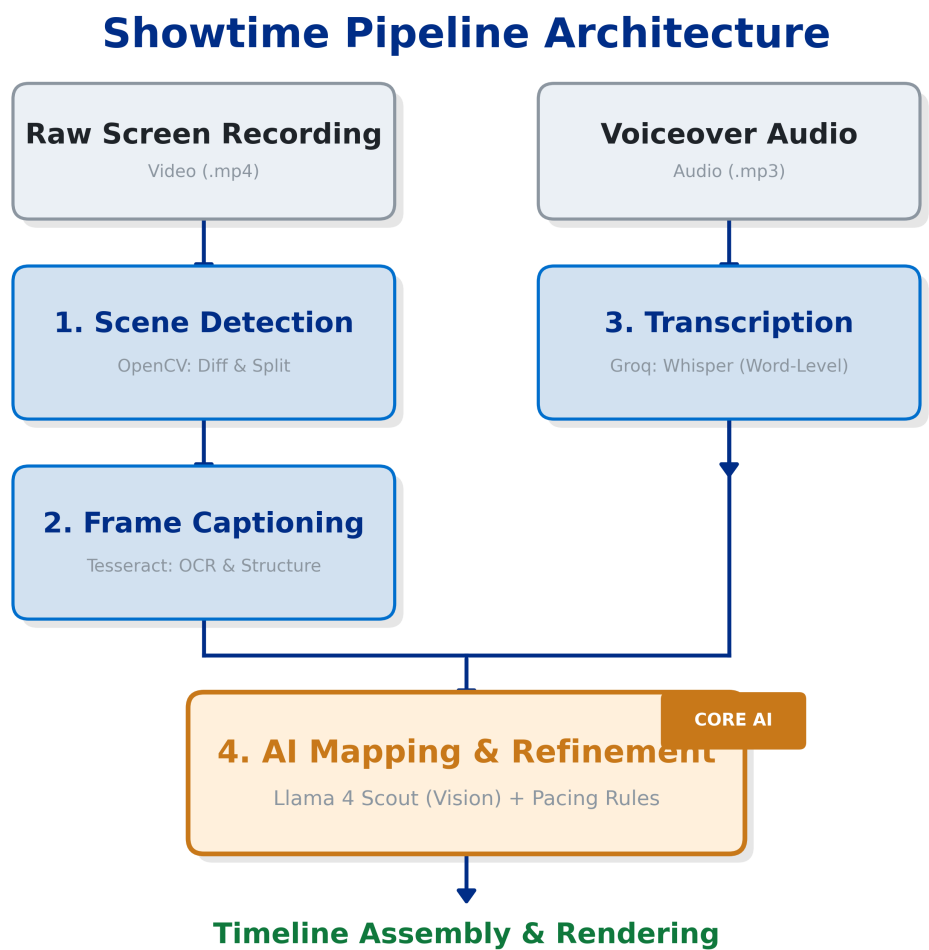


Showtime Pipeline Architecture

Showtime's core objective is to take a raw, unedited screen recording and a separate voiceover audio file, and automatically edit them together into a perfectly paced, professional demo video.

Here is a step-by-step breakdown of exactly how the pipeline works:



The 6-Step Processing Flow

1. Scene Detection (`scene_detector.py`)

Goal: Break the raw video down into logical visual segments.

- The pipeline scans the video at 2 frames per second using OpenCV.
- It calculates the pixel difference between consecutive frames to detect "scene changes" (e.g., clicking a new tab, opening a modal).
- **Auto-split & Merge:** If a segment is too long (> 10s), it is scanned again with higher sensitivity to break it down further. If a segment is incredibly short (< 1.5s), it is merged into a neighboring segment to prevent visual flickering.

- **Idle Removal:** If nothing happens on screen for a while (no pixel changes), that segment is marked as `is_idle` and discarded entirely.
- **Keyframe Extraction:** For every valid segment, the exact middle frame is saved as a representative PNG "keyframe".

2. Frame Captioning (`frame_captioner.py`)

Goal: Understand what is happening visually in each segment.

- The keyframe PNG for each segment is analyzed.
- **OCR (Tesseract):** Extracts all readable text on the screen.
- **Structural Analysis:** Analyzes the UI density and layout.
- The output is a rich text description of the screen segment (e.g., "Screen showing GitHub repository with a green 'Code' button").

3. Audio Transcription (`audio_analyzer.py`)

Goal: Understand the narrative and get exact timestamps for every spoken word.

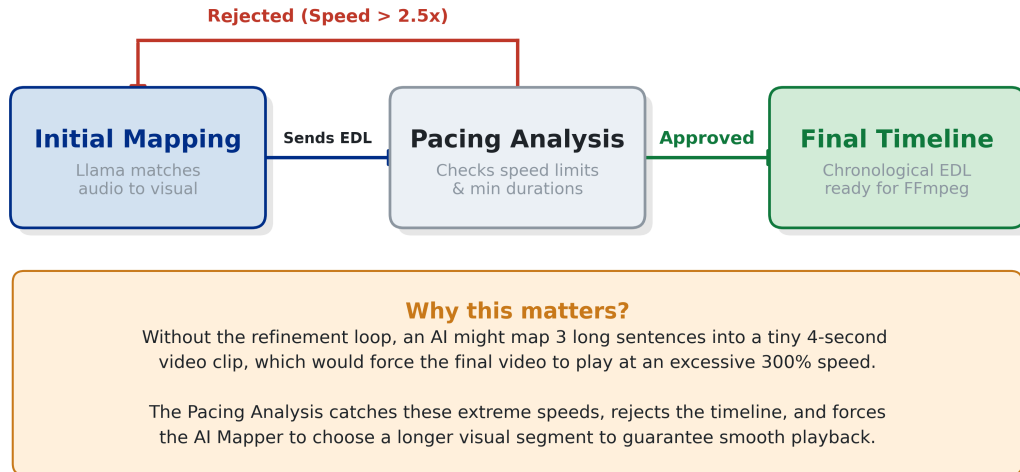
- The voiceover audio file is processed using Groq's `whisper-large-v3-turbo` model.
- It provides a highly accurate transcript with **word-level timestamps**.
- The words are grouped together into localized "Voiceover Sentences" (e.g., Sentence 1: 0.5s - 4.2s: "Welcome to our new app.").

4. AI Mapping (`ai_mapper.py`)

Goal: Determine which visual segment should be shown on screen while the narrator is speaking a specific sentence.

- This is the most complex step, using a 3-tier fallback strategy and a refinement loop:
 1. **Vision Mapping (Llama 4 Scout):** The AI is actually shown the keyframe images alongside the parsed voiceover sentences. It intelligently pairs the sentences to the visuals (e.g., pairing "I'll click the settings gear" with the keyframe that actually shows the settings page).
 2. **Text Fallback (Llama 3.3 70B):** If the vision model fails, the text model uses the OCR descriptions from Step 2 to do the mapping.
 3. **Chronological Fallback:** If AI fails entirely, it blindly maps sentences to segments proportionally based on time.
- **Action Assignment:** For each sentence, the AI also decides if the video should `PLAY` (because an action is happening) or `FREEZE` (because the user needs time to read a static screen).
- **Refinement Loop:** The mapping is analyzed for pacing. If a video clip will be forced to play too fast (> 2.0x speed) or freeze for too long (> 8s), the AI is warned and given a chance to re-map the sentences to longer visual segments to improve the final pacing.

AI Mapping Refinement Loop



5. Timeline Assembly (`timeline.py`)

Goal: Translate the AI's logical mapping into a frame-accurate Edit Decision List (EDL).

- The pipeline calculates exactly which milliseconds of the source video will be used for each voiceover sentence.
- **Speed Constraints:** It enforces a minimum video duration (1.0s) per clip so that no clip flashes on screen too quickly.
- **Gap Insertion:** If there is a natural pause in the voiceover (e.g., the narrator stops talking for 2 seconds), the timeline inserts a "Gap Clip" where the video continues to play naturally in silence, rather than freezing awkwardly.

6. Video Rendering (`renderer.py`)

Goal: Generate the final MP4 file.

- The timeline is passed to FFmpeg to actually cut and paste the video together.
- For each clip in the timeline, the video is trimmed to match the exact duration of the voiceover audio it's paired with.
- **Adaptive Speed:** If the video segment is longer than the voiceover audio, it is sped up. If it is shorter, it is slowed down.
- **Auto-Freeze:** If a clip would need to be sped up beyond `2.5x` speed to fit (which looks choppy and terrible), the renderer catches this and automatically switches to `FREEZE` mode, holding a single keyframe still while the audio plays.
- All individual clips (content, freezes, and gaps) are seamlessly concatenated together into the final `.mp4` output.

Timeline Rendering: Clip Types

